

A Fuzzy Logic Controller for an Assistive-Care Flat: Design, Genetic Tuning of Its Membership Functions, and an Optimiser Comparison on CEC'2005

Bibek Paudyal (250288)

Sajan Mahat (250289)

Siddhartha Bhatta (250620)

ST7085CEM Advanced Machine Learning

Abstract—We design a fuzzy logic controller that manages heating, cooling and lighting in one room of an assistive-care flat for a resident with limited mobility. Its 54 rules are written in plain linguistic terms so that a carer can check them, and the controller is implemented in both Python and MATLAB, which produce the same outputs. A genetic algorithm then tunes the controller, reducing its normalised test error by 32.6% and clearly outperforming random search given the same effort. The tuning came at a cost, however: three of five runs scrambled the order of the fuzzy sets, so some rules no longer meant what they said, and four of five left parts of the input range with no output at all; no run avoided both problems. Finally, a genetic algorithm, particle swarm optimisation and simulated annealing are compared on two standard benchmark functions. None wins everywhere: particle swarm is best in two dimensions, the genetic algorithm is the most reliable in ten, while the ranking of simulated annealing depends on the statistic used.

Index Terms—fuzzy logic control, Mamdani inference, membership function tuning, genetic algorithms, particle swarm optimisation, simulated annealing, CEC'2005, assistive technology

I. INTRODUCTION

Assistive-care housing exists so that residents whose disability limits mobility can live independently. Environmental control is central to that: someone who cannot readily cross a room to reach a thermostat or a light switch, or who cannot quickly add or remove a layer of clothing, depends on the room getting it right without being asked. This paper designs a fuzzy logic controller for the two environmental services where that dependence is sharpest — thermal comfort and lighting — tunes its membership functions with a genetic algorithm, and benchmarks three optimisation techniques against each other on standard test functions.

Fuzzy control suits the problem for a specific reason. Zadeh's fuzzy sets [1] let a controller be specified in the vocabulary a domain expert already uses — “the room is cool”, “the resident is resting” — rather than in a transfer function. Where the specification originates as human judgement, and where somebody other than an engineer must approve the resulting behaviour, that is an engineering advantage rather than a presentational one.

A. Scope and contribution

The controlled space is one room of an assistive-care flat. Four sensor inputs drive two actuators through 54 rules. The contribution is not a new inference algorithm; it is a complete, verified design together with an honest account of what happens when that design is handed to an optimiser. Three results are worth stating in advance:

- The controller is implemented twice, independently, and the two implementations are shown to agree — so the

reported behaviour is a property of the design rather than of one program.

- Genetic tuning improves accuracy by 32.6% and decisively beats a budget-matched random search, but it silently destroys two structural properties of the controller that were never written into its objective.
- On the benchmark functions no optimiser dominates, and the cell with the largest apparent difference between two algorithms proves statistically insignificant once the individual runs are examined rather than their means.

B. Why activity and preference are inputs

Two of the four inputs are not the obvious sensors, and both follow from the assistive-care setting.

Activity matters because thermoregulation is not uniform across residents. Reduced mobility means reduced metabolic heat production, and several conditions affecting mobility — spinal cord injury in particular — also impair vasomotor and sweating responses below the level of the lesion. A resting occupant therefore needs a *warmer* room than a general-purpose thermostat would choose, and the shift is large enough to warrant a first-class input rather than a fixed offset.

Preference matters because comfort is subjective and, in an assistive setting, contested: what a building manager considers efficient and what a resident considers comfortable are different quantities. Exposing preference as an input keeps the resident in the control loop rather than optimising them out of it. It is a standing dial the resident sets, not a transient request.

II. CONTROLLER DESIGN

A. Linguistic variables

Table I gives the four inputs and two outputs. Universes extend past the comfort band in both directions so that a fault condition — a heating failure in winter, solar gain in summer — remains inside the modelled range rather than being clipped. Section V-F shows this to be the first property genetic tuning destroys.

The HVAC output is a single *signed* actuator rather than separate heating and cooling channels. The two are physically exclusive, and a controller able to command both at once can fight itself; a signed axis makes that state unrepresentable rather than merely discouraged.

B. Inference model: Mamdani rather than Sugeno

Mamdani inference [2] is used in preference to the Takagi–Sugeno model [3], for three reasons specific to this application.

First, the rule base is elicited from human domain knowledge rather than fitted to logged data. Mamdani's fuzzy

TABLE I

THE SIX LINGUISTIC VARIABLES. UNIVERSES EXTEND PAST THE COMFORT BAND IN BOTH DIRECTIONS SO THAT A FAULT CONDITION — A HEATING FAILURE IN WINTER, SOLAR GAIN IN SUMMER — REMAINS INSIDE THE MODELLED RANGE RATHER THAN BEING CLIPPED AT THE BOUNDARY.

	Variable	Universe	Fuzzy sets	Sensed by
input	room_temp	14–34 °C	Cold, Cool, Comfortable, Warm, Hot	wall thermistor
input	activity	0–10 index	Resting, Light, Active	PIR motion sensor + wrist accelerometer
input	daylight	0–100% of design lux	Dark, Dim, Bright	ceiling lux sensor
input	preference	–5 (cooler) ... +5 (warmer)	Cooler, Neutral, Warmer	resident-set dial, persisted
output	hvac	–100 ... +100% of plant capacity	CoolHigh, CoolLow, Off, HeatLow, HeatHigh	signed actuator command
output	dimmer	0–100% of full lamp output	Off, Low, Medium, High	lamp dimmer command

consequents express that knowledge in the form it arrives in — “if the room is cold and the resident is resting, heat hard” — whereas a Sugeno consequent is a function of the inputs and has no natural reading until it is evaluated.

Second, and decisively in an assistive-care setting, the behaviour must be approved by someone who is not an engineer. An occupational therapist or carer can check the sentence IF room_temp is Cold AND activity is Resting THEN hvac is HeatHigh against their own judgement. The first-order Sugeno equivalent, $hvac = 71.2 - 3.4T + 1.1A + 0.02D + 4.6P$, is not something a domain expert can audit, and after tuning its coefficients carry no particular meaning at all. Interpretability is a functional requirement here, not a nicety.

Third, the plant is slow: room air temperature moves over minutes, so Sugeno’s cheaper arithmetic buys nothing that matters at run time. Where it *would* matter is inside the tuning loop of Section V, and Section V-G quantifies exactly what a Sugeno formulation would change there.

C. Membership functions

Triangles are used for interior sets and trapezoids at the edges of every universe (Fig. 1).

Interior sets need a single unambiguous prototype value, which the apex of a triangle provides: there is exactly one temperature that is most *Comfortable*. Edge sets must instead *saturate*, because 8 °C and 14 °C are not meaningfully different to an occupant and both should command full heat. A trapezoid with $a = b$ expresses that; a triangle cannot, since it would make one extreme of the universe less Cold than a point just inside it. Both shapes are piecewise linear, and so far cheaper to evaluate than Gaussians or generalised bell curves — immaterial for a single inference, but not for the tuning loop, which evaluates the controller millions of times.

Adjacent sets overlap at roughly half the base width, crossing near $\mu = 0.5$ [4]. This buys two properties, both verified rather than assumed in Section III: the control surface stays continuous, and no input value falls through the rule base.

D. Rule base

The controller has **54 rules**: 45 thermal rules over room_temp $\times$ activity $\times$ preference, and 9 lighting

rules over daylight $\times$ activity.

They are generated from additive fuzzy associative memory (FAM) tables [5] rather than enumerated individually. Each antecedent contributes an integer shift on a five-point demand scale; the shifts are summed and the result saturated back into range:

$$m = \text{clip}(s_{\text{act}} + s_{\text{pref}}, -1, +1), \quad (1)$$

$$d = \text{clip}(b_{\text{temp}} + m, -2, +2), \quad (2)$$

with the demand d mapped to {CoolHigh, CoolLow, Off, HeatLow, HeatHigh}.

The three lookup tables are small. Temperature contributes a base demand $b_{\text{temp}} \in \{+2, +1, 0, -1, -2\}$ running from Cold to Hot; activity contributes $s_{\text{act}} = +1$ for Resting, 0 for Light, -1 for Active; preference contributes $s_{\text{pref}} = -1, 0, +1$ for Cooler, Neutral, Warmer. To take one rule as a worked example, a Cool room ($b_{\text{temp}} = +1$) with an Active occupant ($s_{\text{act}} = -1$) at Neutral preference ($s_{\text{pref}} = 0$) gives $m = \text{clip}(-1, -1, +1) = -1$ and $d = \text{clip}(1 - 1, -2, +2) = 0$, which maps to Off. That is rule R17, and it is one of the four that fires in the worked scenario of Section IV.

The additive structure makes the rule base *auditable*. A carer can be shown three small tables (Fig. 2) instead of a 45-row list, and can check a claim such as “resting shifts demand one step towards heating” on its own, rather than having to trust 45 independently asserted rules to be mutually consistent. It also makes the rule base cheap to revise: changing the metabolic assumption about resting occupants means editing one number, not auditing fifteen rules for consistency afterwards.

A second property follows from the arithmetic rather than from inspection. Because b_{temp} is monotone in temperature and the modifier m does not depend on temperature at all, the demand d is monotone in temperature for every fixed activity and preference. The controller therefore cannot respond to a colder room by heating less — not merely does not, on the states that were checked, but cannot, for any of the 45 combinations. Monotonicity of this kind is normally an emergent property one has to test for; here it is a consequence of how the rule base is constructed.

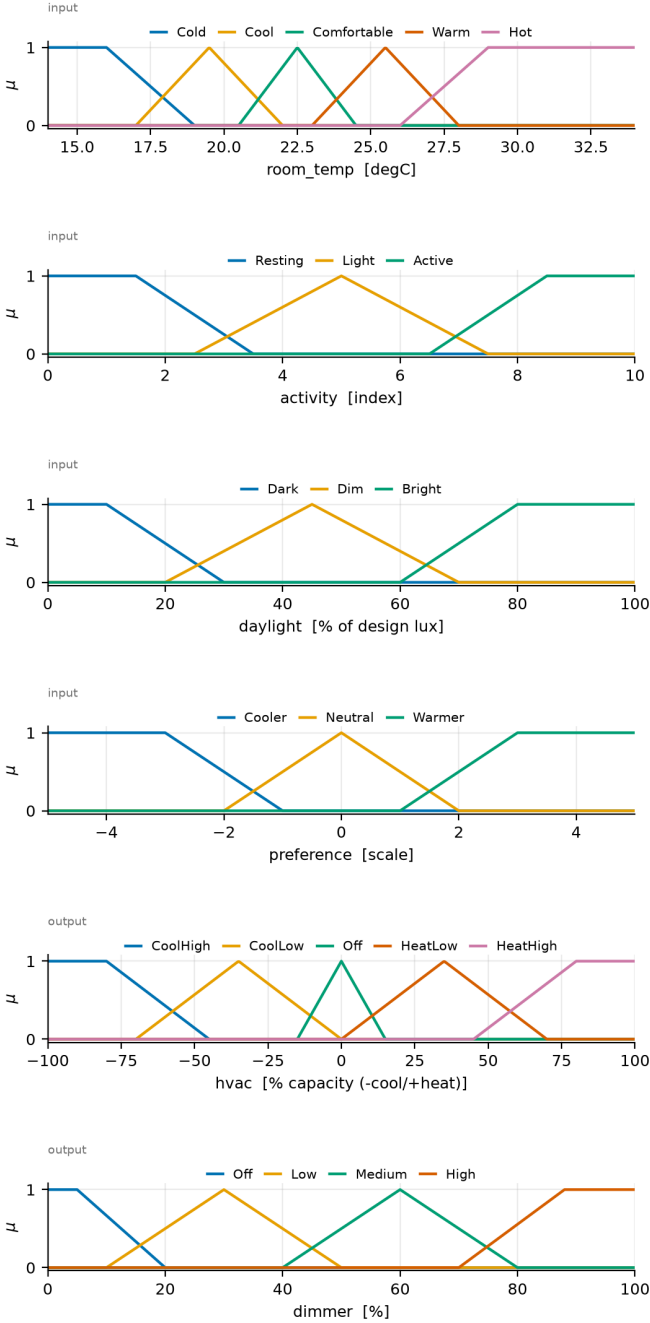


Fig. 1. Membership functions for all six variables. Triangles inside, trapezoids at the edges so the extremes saturate; adjacent sets cross near $\mu = 0.5$.

The inner clip in (1) carries a safety property. Without it, activity and preference each contribute a full step and can therefore *overrule* the temperature reading rather than adjust it: a Comfortable 22.5 °C room with a resting occupant who preferred warmth would sum to +2 and command *HeatHigh*, driving that room towards 30 °C. Clamping the combined modifier to a single step guarantees that the measured temperature sets the demand to within one step, so the controller can never heat a Hot room or cool a Cold one. That property holds by construction, for every one of the 45 thermal rules,

rather than by inspection.

The lighting rules follow the same principle on a four-point scale: activity sets the illuminance the task needs, and available daylight is subtracted from it, since the lamp need only make up the shortfall. **One entry departs from the arithmetic.** A resting occupant in partial daylight (Dim) computes to needing no lamp at all. Falls are the dominant injury risk in this population and a principal reason assistive-care housing exists, so the controller maintains a floor of orientation lighting whenever daylight is not Bright: (Dim, Resting) $\rightarrow$ Low rather than Off. This trades a small amount of energy for a safety margin, and is the one place where the rule base is not a pure function of the FAM arithmetic.

E. Inference and defuzzification

Conjunction is $\min$, implication is $\min$ (clipping), aggregation is $\max$ [2], [4]. Each of the three is a choice, and each was made for a stated reason rather than taken as a default.

AND as $\min$ rather than prod . The minimum t-norm makes a rule's firing strength equal to its weakest satisfied antecedent, which is the reading a domain expert intends by "and": a rule about a cold room with a resting occupant should fire to the degree that the *least* well satisfied of those two conditions holds. The product t-norm instead multiplies, so three antecedents each satisfied to degree 0.5 would fire at 0.125 rather than 0.5. With three antecedents on every thermal rule, that would systematically suppress the rules that fire in the overlap regions — precisely the regions where the controller is interpolating and where its output matters most.

Implication as clipping rather than scaling. A clipped consequent retains a flat top at its rule's firing strength, which keeps each rule's contribution individually visible in the aggregated set — the plateaus in Fig. 10 can each be attributed to a named rule. Under scaling the contributions blend into one another and the aggregate becomes difficult to read, losing the property that motivated Mamdani inference in the first place.

Aggregation as $\max$. Union rather than sum, so that two rules recommending the same action do not produce a stronger recommendation than either alone. With an additive FAM this matters: several rules routinely share a consequent, and under summation their agreement would inflate the aggregated set and bias the centroid, turning "two rules agree" into "act twice as hard".

The controller defuzzifies by centroid. All three standard candidates were implemented and compared on measured behaviour rather than convention (Fig. 3). Centroid uses every rule's contribution, which is the point of having run the inference, and moves smoothly as the inputs move. The bisector is cheaper and less sensitive to a long thin tail, but advances in steps as the equal-area crossing jumps between grid cells — behaviour an actuator should not be asked to follow. Mean-of-maximum reaches further towards the ends of the universe but ignores every set that is not the strongest, discarding most of the information the rule base has just produced, and jumps as the dominant rule changes [6].

Rule base as fuzzy associative memory tables: 45 thermal + 9 lighting = 54 rules

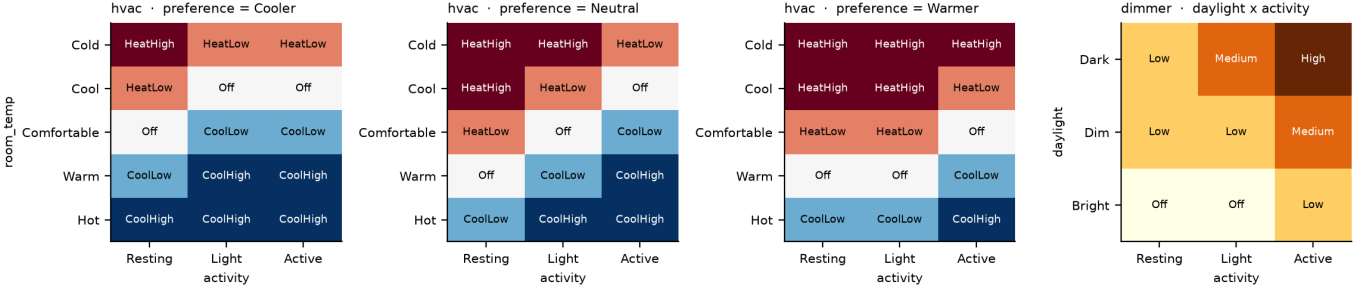


Fig. 2. The complete rule base as FAM matrices: three temperature $\times$ activity slices, one per preference setting, plus the lighting table. Colour is the signed demand step, so the monotonicity the additive structure guarantees appears as a gradient that never reverses along either axis — the controller cannot respond to a colder room by heating less. Saturation is visible at the corners: Cold + Resting + Warmer and Cold + Resting + Neutral both command HeatHigh, because once demand is at full heat there is nowhere further to go.

The comparison also quantifies a real cost of that choice, discussed in Section IV: centroid cannot command the full actuator range.

III. IMPLEMENTATION AND VERIFICATION

A. Two independent implementations

The controller exists in two forms: a purpose-written Python Mamdani inference engine and a custom base-MATLAB implementation. The MATLAB controller definition is generated from the same variables, rules and membership functions. This reduces transcription errors; agreement must still be checked numerically.

This yields a genuine verification opportunity. The two are identical in *definition* but independent in *arithmetic*: different implementations of fuzzification, clipping, aggregation and centroid integration. Agreement between them is therefore evidence that both are correct, and disagreement would localise a defect to one of them. The inference engine is separately validated against an established open-source fuzzy library, which must reproduce every consequent activation to within 10^{-12} and must converge on the same crisp output as the output universe is refined.

Two design decisions in the MATLAB implementation are worth stating because they are the ones that make the comparison meaningful rather than approximate.

The MATLAB system is generated, not transcribed. Maintaining two hand-written definitions of one controller and hoping they stay in step is exactly the kind of silent drift that invalidates a cross-implementation check: the two could eventually differ, and the difference could be mistaken for a finding. Regeneration and cross-implementation checks reduce this risk.

Mamdani inference is implemented directly rather than called from a library. The controller is expressed as a plain data structure and the five inference stages are evaluated explicitly. This was originally a constraint — the MATLAB licence available for this work does not include the fuzzy inference library, so those functions were unavailable. The custom engine exposes firing strengths, clipped consequents and aggregated sets to support the worked examples. Plots are

TABLE II
STRUCTURAL AUDIT OVER A 70,875-POINT INPUT GRID. EACH ROW IS A PROPERTY THE DESIGN INTENDS, MEASURED RATHER THAN ASSUMED.

Property	Measured
Inputs with no defined output	0
Dead rules (never fire)	0
Rules active per input, mean	2.89
Rules active per input, max	12
Inferences per second	15,729
Centroid, exact vs. discrete sum	3.6×10^{-15}

derived from these intermediate values and checked against the numerical trace. This choice does not constitute use of the named Toolbox.

B. Base-MATLAB implementation

Figures 4, 5 and 6 show the MATLAB visualisations: membership functions, rule inference for a worked scenario, and control surfaces. They are not evidence of a Fuzzy Logic Toolbox implementation. The saved code uses a custom engine because that toolbox was unavailable on the project's licence. The brief permits a simulated system programmed in MATLAB, FuzzyLite or Juzzy and provides Toolbox reference links. The base-MATLAB implementation follows that programming-language option without claiming to use the Toolbox.

The evaluation reports a direct numerical comparison against the reference engine. Both custom implementations use 501 output samples: unequal grids could change centroid estimates independently of inference correctness. The recorded base-MATLAB code check was executed in GNU Octave, with approximately 3.6×10^{-7} difference on the worked scenario, within the 10^{-3} tolerance on both actuators. This is separate from the Python/scikit-fuzzy tests.

C. Structural audit

The controller was swept over a 70,875-point grid covering the whole input space (Table II).

Zero uncovered inputs and zero dead rules are the two properties the overlap choice was made to buy, and both hold.

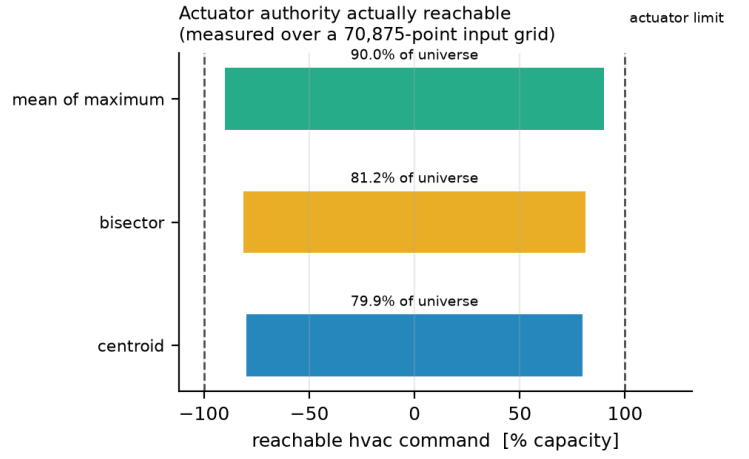
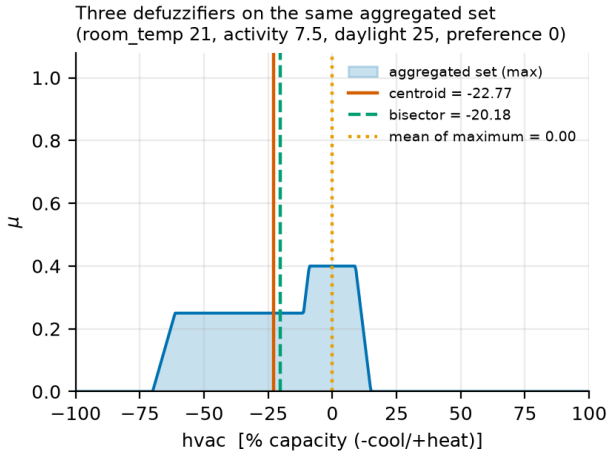


Fig. 3. Defuzzification compared on measured behaviour. **Left:** the three methods applied to the same aggregated set. **Right:** the range of HVAC commands each can actually reach, measured over a 70,875-point input grid. Centroid spans only 79.9% of the $\pm 100\%$ universe against mean-of-maximum's 90.0% — a real limit on actuator authority, and the price paid for smoothness.

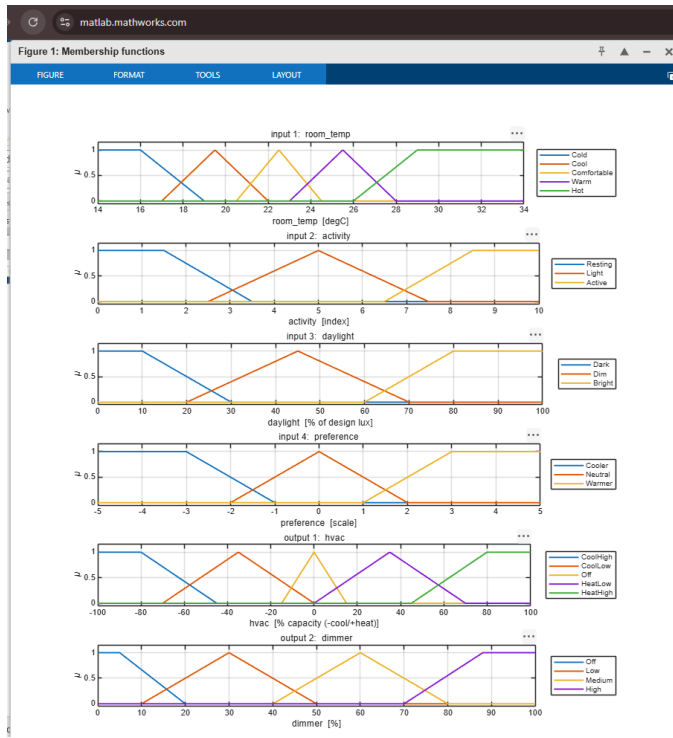


Fig. 4. MATLAB visualisation: membership functions for all four inputs and both outputs. Compare with Fig. 1, produced independently by the reference engine.

Figure 7 shows the distributions behind those two counts. The left panel is the number of rules firing simultaneously: it has no zero bin, so no input state falls through the rule base. The right panel is how often each of the 54 rules fires across the space; no bar is zero, so no rule is dead. Both are consequences of overlapping adjacent sets near $\mu = 0.5$, and both are the kind of claim that is cheap to assert and easy to get wrong, so they are measured rather than assumed.

The distribution is also informative about the cost of infer-

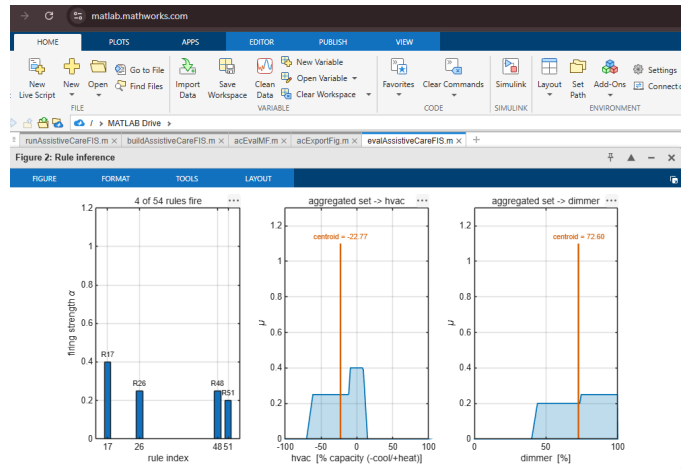


Fig. 5. MATLAB rule inference for the worked scenario (room_temp = 21, activity = 7.5, daylight = 25, preference = 0): the firing strengths, and the aggregated output set with its centroid for each actuator.

ence. A mean of 2.89 simultaneously active rules out of 54 means the controller is doing very little work per decision: a reading typically lands in two sets on temperature and one or two on the other variables, and the rule base is sparse enough that only a handful of combinations survive. The maximum of 12 occurs where a reading sits on a crossover point of three antecedent variables at once. That sparsity is what makes 15,729 inferences per second achievable without any optimisation, and it is a direct consequence of the overlap choice rather than of implementation effort.

The final row of Table II confirms that the integration scheme introduces no error of its own: exact trapezoidal integration of the aggregated set and a discrete centre-of-gravity sum over the same grid agree to 3.6×10^{-15} on the HVAC output, which is machine precision. Any disagreement between this controller and another implementation is therefore attributable to the model, not to the quadrature.

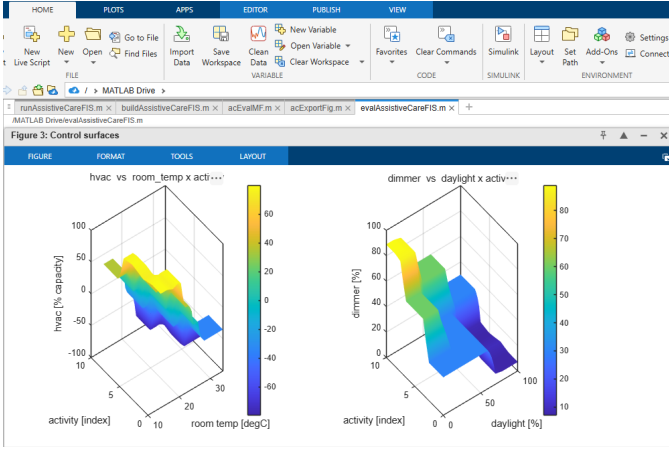


Fig. 6. MATLAB control surfaces for both actuators. Compare with Fig. 11.

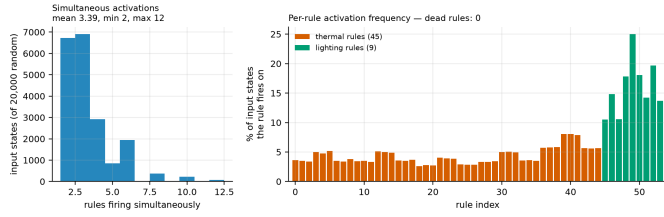


Fig. 7. Rule-base coverage over 20,000 uniformly sampled input states. **Left:** how many rules fire simultaneously — the distribution has no zero bin, so no input state falls through the rule base. **Right:** how often each of the 54 rules fires, thermal rules and lighting rules distinguished; no bar is zero, so there are no dead rules.

IV. ANALYSIS OF OUTPUT BEHAVIOUR

A. A worked inference, stage by stage

Consider an evening scenario: the resident is moving about preparing a meal as the light fades. $\text{room_temp} = 21^\circ\text{C}$, $\text{activity} = 7.5$, $\text{daylight} = 25\%$, $\text{preference} = 0$.

Fuzzification (Fig. 8) gives $\mu_{\text{Cool}} = 0.40$ and $\mu_{\text{Comfortable}} = 0.25$ on temperature, $\mu_{\text{Active}} = 0.50$ on activity, $\mu_{\text{Dark}} = 0.25$ and $\mu_{\text{Dim}} = 0.20$ on daylight, and $\mu_{\text{Neutral}} = 1.0$ on preference.

Rule activation (Fig. 9) fires four of the 54 rules. R16 ($\text{Cool} \wedge \text{Active} \wedge \text{Neutral} \rightarrow \text{Off}$) at $\alpha = 0.40$ and R25 ($\text{Comfortable} \wedge \text{Active} \wedge \text{Neutral} \rightarrow \text{CoolLow}$) at $\alpha = 0.25$ carry the thermal decision; two lighting rules carry the lamp.

Implication and aggregation (Fig. 10) clip each consequent set at its rule's firing strength and take the union.

Defuzzification returns $\text{hvac} = -22.77\%$ — gentle cooling, since the occupant is generating heat in an already-adequate room — and $\text{dimmer} = 72.60\%$, substantial light for a task in falling daylight. Both are the actions a carer would expect, which is the qualitative check that matters most.

B. Control surfaces

A control surface is the whole input–output map of the controller in one picture, and it is where design intent either shows up or does not. Figure 11 gives the HVAC actuator against temperature and activity; Fig. 12 gives the dimmer

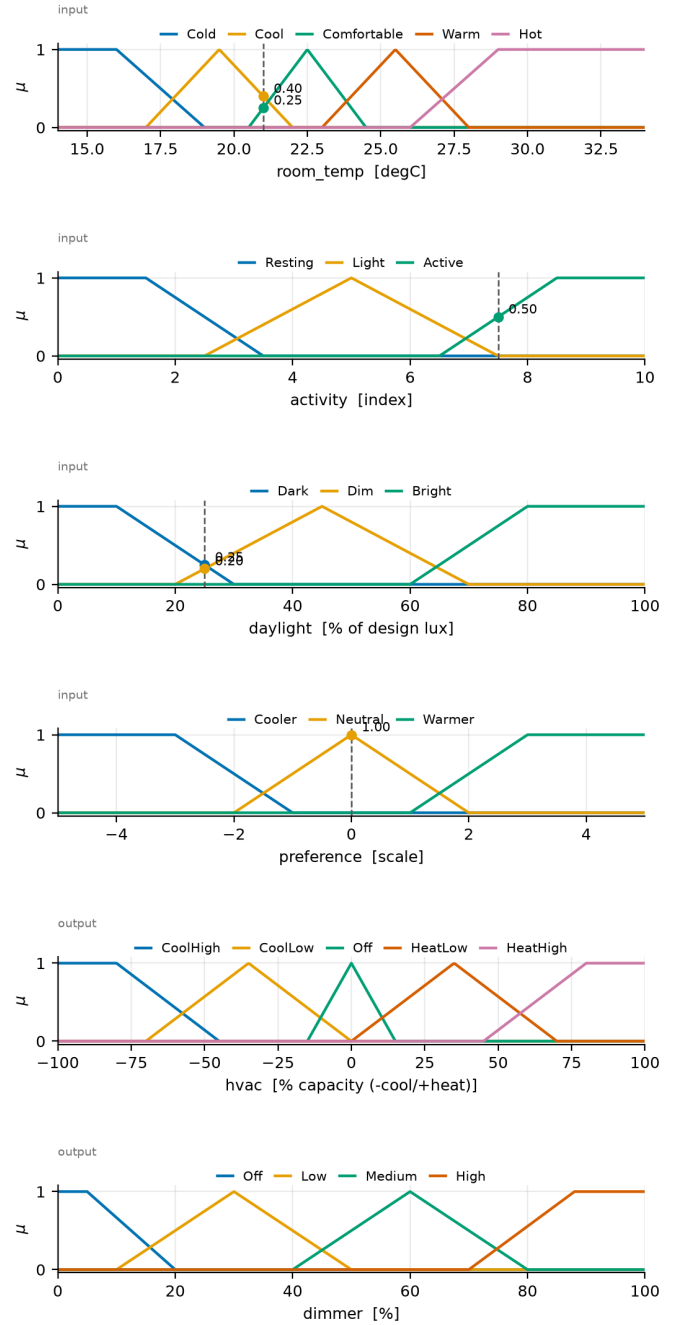


Fig. 8. Fuzzification of the worked scenario: each crisp reading dropped onto its variable's sets, with the resulting degrees of membership marked.

against daylight and activity. Figure 22 in the appendix gives the third slice, HVAC against temperature and preference.

Four intended properties are visible, and each corresponds to a design decision made in Section II.

Temperature dominates. The HVAC surface falls monotonically from full heat to full cool as the room warms, and no combination of activity and preference reverses that. This is the clamped modifier of (1): because the combined activity-plus-preference shift is capped at one step on a five-step scale, the measured temperature always sets the demand to within

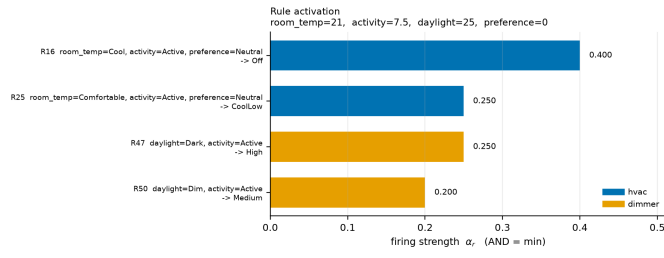


Fig. 9. Rule activation for the worked scenario: 4 of 54 rules fire, with their firing strengths. Sparse activation is the intended consequence of overlapping adjacent sets at $\mu \approx 0.5$ rather than more widely.

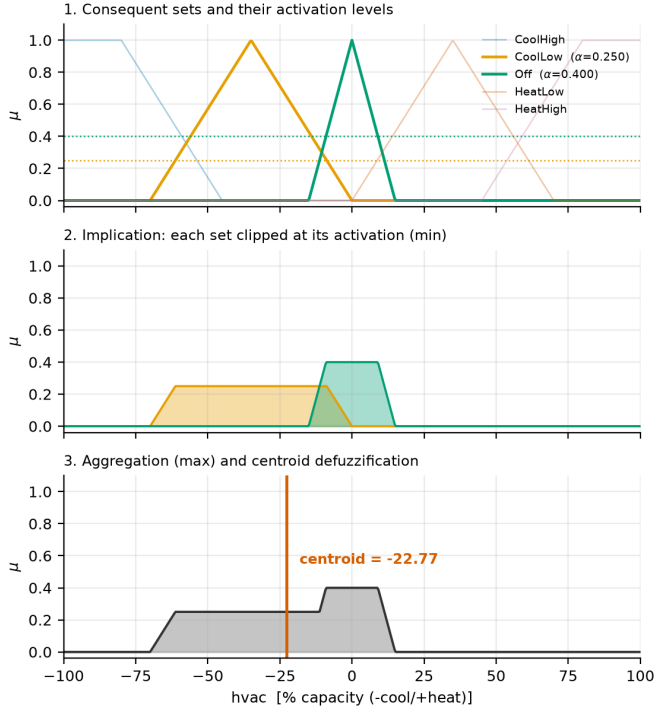


Fig. 10. Implication, aggregation and defuzzification for hvac. Each consequent set is clipped at its rule's α (the flat tops), the union is taken, and the centroid of the result is the commanded value. The same three stages for the dimmer output are in Fig. 21 in the appendix.

one step. Monotonicity here is a structural guarantee of the rule base rather than an emergent property that happened to hold.

Activity and preference shift the surface without tilting it. Moving along the activity axis translates the whole temperature response rather than changing its slope, which is exactly the intended semantics: a resting occupant should be given a warmer room, not a differently-shaped response. Figure 23 in the appendix shows the same effect as three overlaid cross-sections, where the ordering is easier to read off than from a three-dimensional surface.

The surface is continuous, with no cliffs. Adjacent sets overlap near $\mu = 0.5$, so as a reading crosses from one set into the next the two consequents blend rather than switch. An actuator can follow this; it would not be able to follow a staircase. The visible terracing is the five-level demand scale,

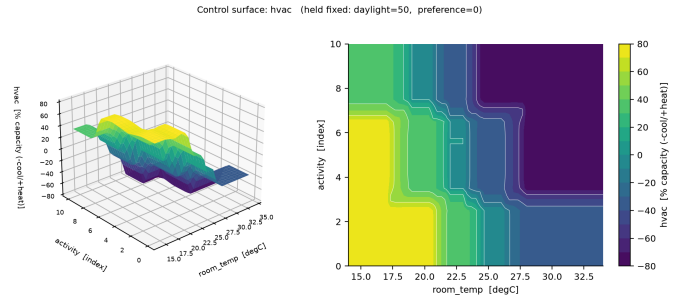


Fig. 11. Control surface: hvac against room_temp and activity, at neutral preference. Monotone in temperature, shifted but not tilted by activity, and continuous throughout. The saturation plateaus at roughly $\pm 79.9\%$ are the centroid authority limit discussed below.

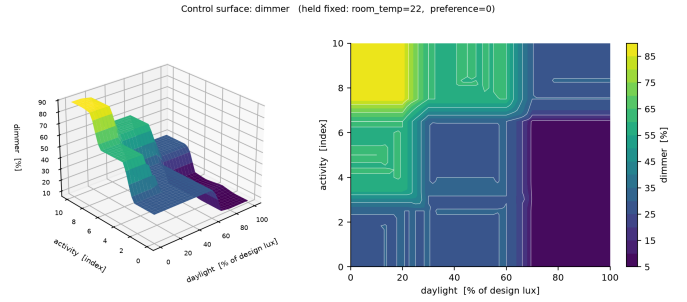


Fig. 12. Control surface: dimmer against daylight and activity. The lamp makes up the shortfall between what the task needs and what daylight supplies. The floor at low output is the fall-risk override: orientation lighting is never fully extinguished while daylight is below Bright.

softened by the overlap into ramps rather than steps.

The dimmer surface makes up a shortfall. The lamp output falls as daylight rises and rises with activity, which is the subtraction the lighting FAM performs. The flat floor at low output is the fall-risk override: the lamp never switches off entirely while daylight is below Bright, which is visible as a plateau that does not descend to zero.

C. An operational day

A surface shows what the controller *can* do; a time series shows what it actually does in the states a flat occupies. Figure 13 runs a 24-hour profile through the controller — a cold night, a resident waking and transferring, a mild sunlit afternoon, an active evening as daylight fades.

Over the simulated day the HVAC command spans $[-70.97, +76.60]\%$ and the dimmer $[7.0, 87.69]\%$, with 12.8 heating hours and 6.5 cooling hours. Three behaviours the design intends are visible in the traces. The lamp never reaches zero, which is the fall-risk floor holding across every state the day visits. The HVAC command never changes sign abruptly: it passes through the Off region rather than switching between heating and cooling, which is the behaviour a real plant needs if it is not to short-cycle. And the morning transfer — a brief burst of high activity in a cool room — reduces the heat demand rather than raising it, because the occupant is generating metabolic heat, which is the assistive-care-specific behaviour that motivated making activity an input at all.

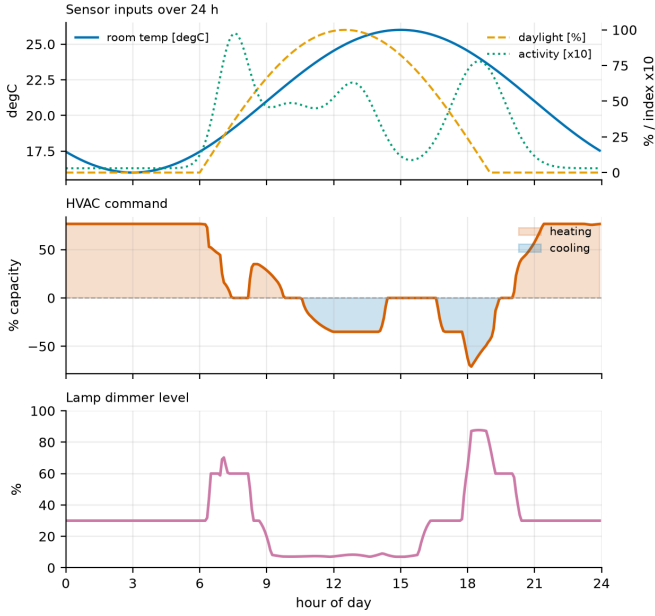


Fig. 13. A simulated operational day. Sensor inputs above, commanded actions below. The dimmer never reaches zero and the HVAC command transitions smoothly rather than switching.

D. Two limitations the analysis exposes

Both of the following emerged from analysing output behaviour rather than from inspecting the design, which is an argument for performing this analysis at all.

The controller cannot reach full actuator authority. Centroid defuzzification spans only $[-79.9, +79.9]$ of the $\pm 100\%$ HVAC universe, because the aggregated set always retains area on the interior side and the centre of gravity is pulled inwards. Measured against bisector (81.2%) and mean-of-maximum (90.0%) in Fig. 3, this is a real limit rather than a plotting artefact: commanding 100% capacity is impossible whatever the inputs. It is acceptable here because the plant is oversized relative to demand and because the alternatives trade it for discontinuity, but a deployment requiring full authority would need to rescale the output universe.

The HVAC response is not pointwise monotone in temperature, by about 0.6% of the output span. Where the demand scale saturates, two adjacent temperature sets share a consequent, and that consequent's activation is V-shaped through their crossover. The effect is bounded and an order of magnitude below any real actuator's resolution.

V. GENETIC TUNING OF THE MEMBERSHIP FUNCTIONS

The controller structure of Section II is now held fixed — the same four inputs, two outputs and 54 rules — and only the membership functions are tuned against a set of input–output examples, following the genetic-fuzzy approach surveyed by Cordón et al. [7].

A. The dataset

The training data is synthetic, and nothing derived from it is evidence about real occupants. No public dataset of

assistive-care environmental preferences exists, and collecting one would require ethical approval and resident participation. The examples are generated instead from an explicitly stated analytical reference policy — saturating linear ramps on set-point error and on task illuminance — with Gaussian noise at 3% of each output's span representing annotator disagreement. 400 examples are drawn and split 280 training / 120 held-out test.

Two properties of that construction matter. Inputs are sampled *uniformly* rather than from a plausible occupancy distribution: a controller tuned only on the states a flat usually occupies would leave the membership functions covering rare states — a heating failure, a heatwave — entirely untuned, and those are the states where the controller matters most. And the reference policy is deliberately a *different functional form* from the controller. Were the target generated by a Mamdani system with the same rule base, the error could in principle be driven to zero, demonstrating only that the encoding can represent its own output. Fitting a genuinely different shape is the harder test, and residual error at convergence is therefore expected rather than a failure.

Accordingly, the improvement reported below establishes that the search can recover a known target. It does not establish that the tuned controller would serve real residents better.

B. Problem encoding and chromosome length

Encoding is **binary**: one gene of 8 bits per tunable membership-function parameter, decoded linearly onto the physical range of the variable it belongs to,

$$v = \ell + \frac{b}{255}(h - \ell), \quad (3)$$

so a room-temperature breakpoint decodes somewhere in $[14, 34]$ degrees and an HVAC breakpoint somewhere in $[-100, +100]$ percent. A fixed word length makes the chromosome length a stated, checkable quantity [8].

The parameter layout, and hence the **chromosome length**, is:

room_temp	17	hvac	17
activity	11	dimmer	14
daylight	11		
preference	11		
81 parameters $\times$ 8 bits = 648 bits			

The resolution this buys is $(h - \ell)/255$: about 0.078°C on room temperature and 0.78 percentage points on the HVAC command, finer than either sensor or actuator resolves. Quantisation is therefore not what limits the result, though it is not free: encoding and decoding the hand-designed controller displaces its parameters by at most 0.392 units and costs +0.000342 in normalised error before the search gains anything — roughly 0.7% of what the search subsequently recovers. Figure 25 in the appendix breaks that cost down per gene.

Infeasible chromosomes are repaired, not rejected. A decoded chromosome routinely proposes a triangle whose parameters are out of order. Discarding the individual would waste the evaluation and bias the search towards whatever

region happens to be feasible, so each membership function’s parameters are sorted ascending on decode. Every point in the 648-bit space therefore maps to a valid controller, and selection pressure stays on fitness.

C. Genetic operators

Roulette-wheel (fitness-proportionate) selection with elitism 2; one-point crossover at $p_c = 0.7$; per-bit mutation at $p_m = 0.01$; population 60 over 150 generations, giving 9,060 evaluations per run [8].

The crossover cut may fall anywhere in the chromosome, *including inside a gene*. Cutting only on gene boundaries would make the 8-bit words atomic and prevent the search from refining a single breakpoint by a small amount. Elitism guarantees the best individual survives untouched, so best fitness cannot fall between generations.

The population is **seeded with the hand-designed controller**, which makes “never worse than the expert design” a property of the search rather than a hope. The seeded individual is charged an evaluation like any other.

D. Fitness function

Span-normalised RMSE against the reference data, expressed as a quantity to maximise:

$$\text{fitness} = \frac{1}{1 + \text{nRMSE}}, \quad \text{nRMSE} = \frac{1}{2} \sum_{j=1}^2 \frac{\sqrt{(\hat{y}_j - y_j)^2}}{h_j - \ell_j}. \quad (4)$$

Two choices inside that are worth stating. **Normalising by span** is necessary because the outputs live on different scales: HVAC spans 200 units of signed capacity, the dimmer 100 units of percent. Averaging raw squared error would let HVAC dominate roughly four to one, leaving the lighting membership functions effectively untuned. **Undefined outputs are charged the full span** rather than dropped, so the search cannot improve its score by evolving a controller with holes in it — a provision that Section V-F shows discourages holes without preventing them.

Roulette-wheel selection requires a positive quantity that is larger for better individuals; $1/(1 + e)$ is bounded in $(0, 1]$ and monotone in the error, so it changes which individuals are selected without changing which is best.

E. Results against a budget-matched control

A genetic algorithm that improves on its starting point has not thereby shown that *evolution* did the work: a search that merely samples the space finely can do the same. Separating the two requires spending an identical number of fitness evaluations sampling at random. Both searches are seeded with the expert controller, both are charged 9,060 evaluations per run, both use the same fitness function, and both are repeated over five seeds.

The genetic algorithm improves on the hand design by **32.6%** and beats the control decisively (Table III, Fig. 15).

The convergence traces (Fig. 14) add two details the summary table hides. First, every one of the five seeds improves on

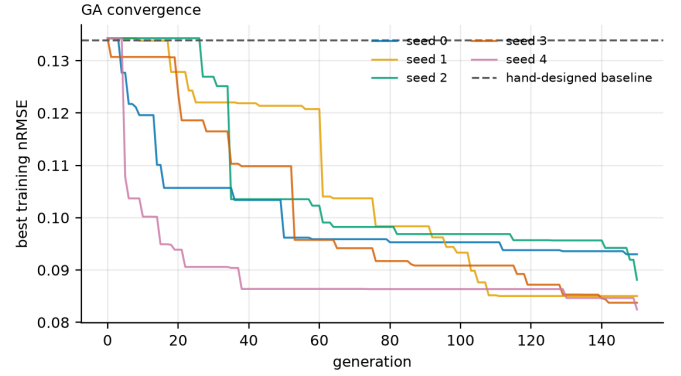


Fig. 14. Convergence of the genetic algorithm over five seeds: best training error against generation, with the hand-designed baseline marked. Every seed improves on the baseline; none has clearly plateaued at the 150-generation budget, which indicates the budget rather than the encoding is the binding constraint.

the baseline, and they do so at markedly different rates — one seed reaches its final quality by generation 40 while another is still descending at generation 140. That spread is ordinary for a stochastic search, and it is the reason the comparison is run over five seeds rather than one. Second, and more usefully, *none* of the five has clearly plateaued by the 150-generation budget. The search is still finding improvements when it is stopped, which says the binding constraint is the evaluation budget rather than the encoding or the operators. A longer run would be expected to do better still; whether that would be a good idea is a separate question, and Section V-F argues it would not.

Figure 16 shows what the tuning did to the membership functions themselves. The sets migrate substantially — this is not fine polishing of the expert design — and on several variables they visibly stop meeting the ends of their universes, which is the defect Section V-F quantifies. Figure 24 in the appendix shows the same improvement in the output space, as predictions against targets on the held-out split.

Random search did not improve on its seed even once. Its result is identical to five decimal places across all five runs, with standard deviation exactly zero, and that value is precisely the expert chromosome it was seeded with. In $5 \times 9,060 = 45,300$ uniform draws from a 648-bit space, not one beat the hand design. The mechanism is plain: the space has 2^{648} points, and uniform sampling of it is hopeless in a way that uniform sampling of a three-parameter space is not. Dimensionality, not sophistication, is what determines whether the evolutionary machinery earns its keep here.

F. What the tuning cost

The 32.6% improvement is real, and reporting it alone would be misleading. Two further audits were run on the tuned controllers, and both fail.

Linguistic ordering breaks in three of five runs. Mamdani inference was adopted because the rule base stays readable, and that argument survives optimisation only if `Cold` remains to the left of `Cool` afterwards. A controller whose sets have

TABLE III

HELD-OUT TEST ERROR OVER FIVE SEEDS. BOTH SEARCHES WERE GIVEN AN IDENTICAL BUDGET OF 9,060 FITNESS EVALUATIONS, AN IDENTICAL EXPERT-SEEDED STARTING POINT AND THE IDENTICAL FITNESS FUNCTION; THEY DIFFER IN EXACTLY ONE RESPECT, WHICH IS HOW CANDIDATES ARE PROPOSED. LOWER ERROR IS BETTER.

Method	mean	std	best	worst	vs. baseline	note
Hand-designed baseline	0.13299	—	—	—	—	the expert design, untuned
Random search (matched)	0.13340	0.00000	0.13340	0.13340	−0.3%	never improved on its seed
Genetic algorithm	0.08964	0.00438	0.08504	0.09682	+32.6%	every seed beat the baseline

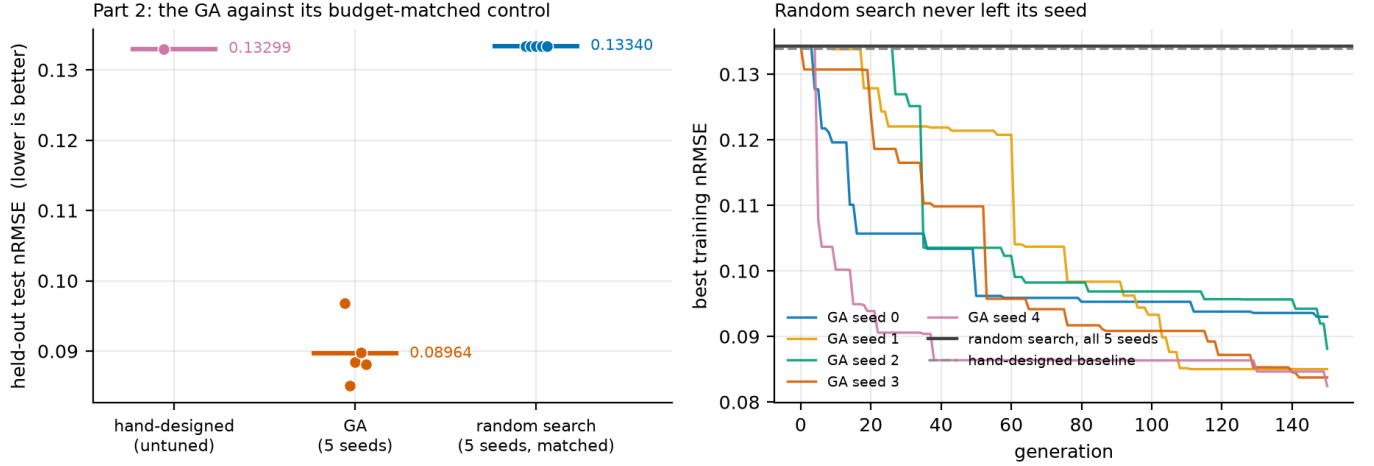


Fig. 15. The budget-matched control. **Left:** held-out test error. The five random-search points coincide exactly, because all five runs returned the expert chromosome they were seeded with. **Right:** convergence against the level random search reached — a flat line, since it never left its seed.

reordered has rules that no longer mean what they say, and the interpretability justification collapses with it. Comparing set centres of mass before and after (Fig. 17), two runs reorder `room_temp` and one reorders `hvac`.

Tuning punches holes in the rule base. The hand-designed controller returns a defined output for every one of 50,000 uniformly sampled input states. Four of the five tuned controllers do not: between 0.37% and 0.80% of states activate *no rule at all*, leaving the aggregated set empty, the centroid undefined, and the actuator without a command (Fig. 18).

The mechanism is precise, and it bears directly on the design rationale of Section II. Trapezoids were placed at the universe edges specifically so the extremes would saturate. After tuning, the outermost sets no longer reach those edges: in one run, *every* input has $\mu = 0$ at both ends of its universe. The states that fall through are therefore exactly the fault conditions — heating failure, heatwave, total darkness, maximum stated preference — that the universes were widened to capture. Unconstrained optimisation removed the property the edge trapezoids existed to provide.

Taken together (Table IV), **no run produced a controller that was both fully covering and linguistically ordered.** The two runs that kept their ordering both lost coverage; the one run that kept full coverage reordered its output sets.

The lesson generalises beyond this controller. The fitness function encodes accuracy and nothing else, so accuracy is what it delivered — at the cost of two safety-relevant structural properties that nobody decided to trade, because nobody had

written them into the objective. Both are cheap to enforce: ordering by extending the repair operator to sort set centres as well as parameters, coverage by a penalty term or by pinning the outermost breakpoints to the universe edges. Neither was applied here, so that the question of whether unconstrained tuning preserves interpretability could be answered by measurement rather than assumed.

G. The solution under a Sugeno model

Under Takagi–Sugeno inference only the *consequent* side changes. The antecedents, the input membership functions and all 54 rules are unaffected, so the 50 input parameters of the chromosome are untouched. What changes is the 31 output parameters, and the range of outcomes is wide (Table V).

Under **zero-order** Sugeno each output fuzzy set becomes a single singleton, so the 5 HVAC and 4 dimmer sets become 9 constants and the chromosome shortens by 27%. Under **first-order** Sugeno each of the 54 rules carries its own linear function of four inputs, so the chromosome grows almost fourfold to 2,560 bits and the search space from 2^{648} to 2^{2560} . Given that 9,060 evaluations against 648 bits does not exhaust the present search, a four-times-longer chromosome increases the search burden at the same budget; its actual performance would require a separate experiment.

The decisive gain is that least squares becomes available. With the antecedents fixed, a Sugeno system is *linear in its consequent parameters*, so the optimal coefficients can be solved directly rather than searched. This enables the

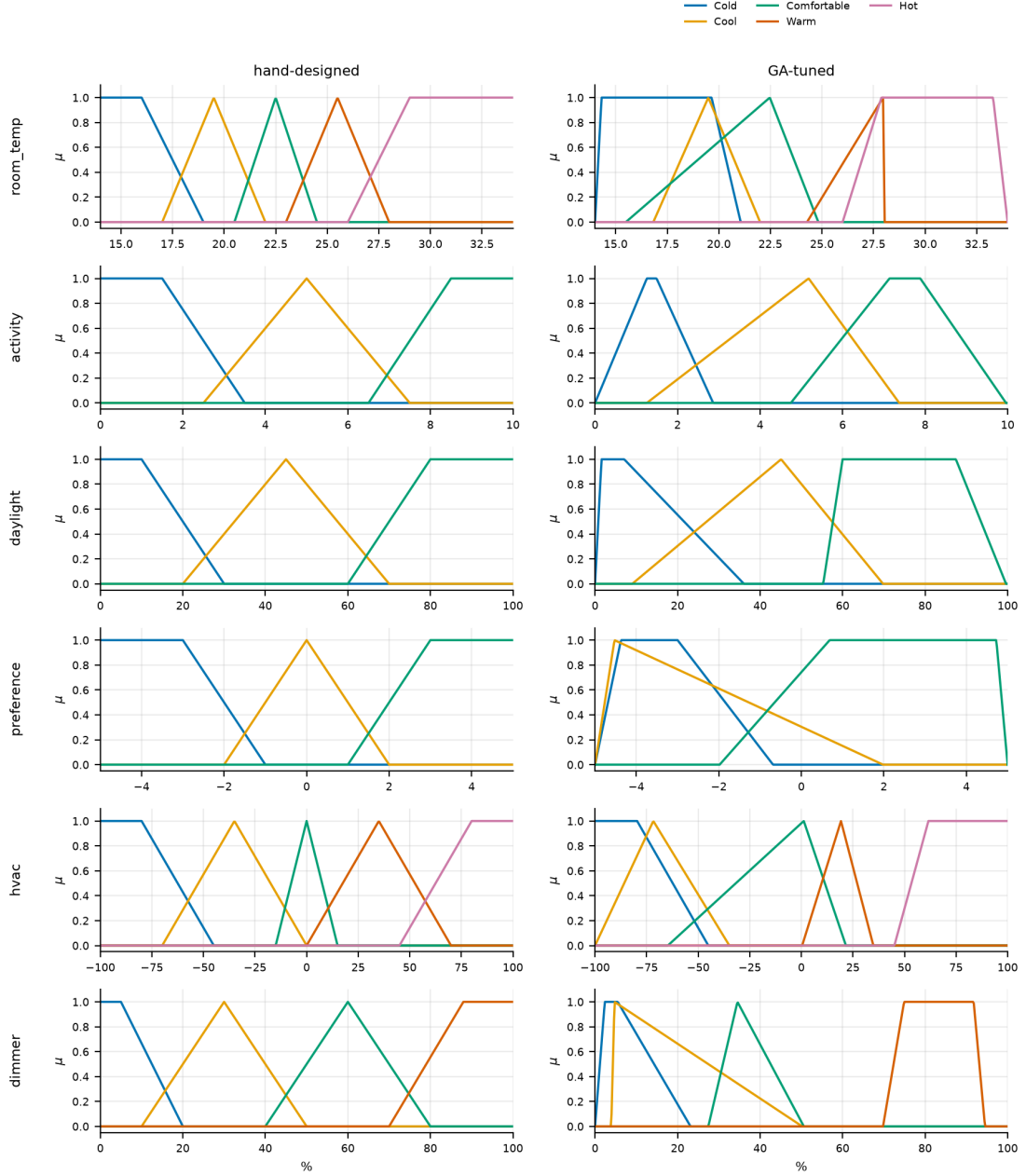


Fig. 16. Membership functions before (left column) and after (right column) genetic tuning, for the best of the five runs. All six variables are shown. The sets migrate substantially rather than being finely polished, and on several variables the outermost sets no longer reach the ends of their universe — the coverage failure quantified in Section V-F. Compare the left column with Fig. 1: the hand design places a saturating trapezoid at each end precisely so that the extremes are fully covered, and that is the property tuning removes first.

hybrid ANFIS-style scheme [9], which would be the approach of choice: the genetic algorithm tunes only the 50 input membership-function parameters (400 bits), and least squares solves the consequents exactly at every fitness evaluation. The search space shrinks by more than a third *and* every individual is scored using optimal consequents. This is precisely why ANFIS is built on Sugeno rather than Mamdani: centroid defuzzification is not linear in the output membership-function parameters, so no equivalent closed-form step exists for the system built here.

Evaluation would also become cheaper — a weighted average over active rules replaces integrating a 501-point aggregated set per output per sample — and the repair operator would apply only to the input half, since consequent coefficients carry no ordering constraint.

What deteriorates is the property for which Mamdani was chosen. The present output-set interpretation and linguistic-ordering audit of Section V-F would no longer apply directly. Sugeno rules can still be inspected, but would require a different account of their output behaviour. **More seriously,**

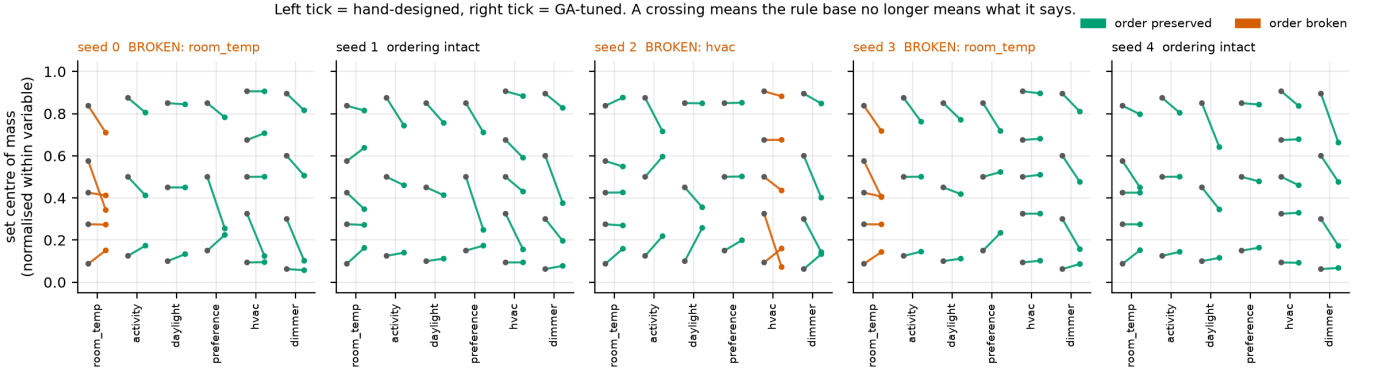


Fig. 17. Does the tuned controller still mean what its rules say? Set centres of mass before (left tick) and after (right tick) tuning, per run. A crossing is a reordering: two runs reorder `room_temp` and one reorders `hvac`.

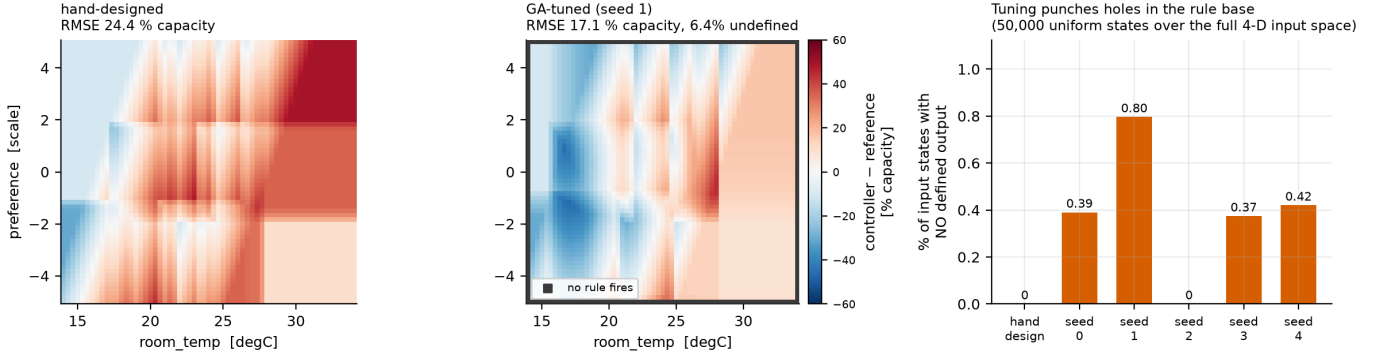


Fig. 18. **Left, centre:** where the controller disagrees with the reference policy, before and after tuning. Tuning shrinks the error band rather than removing it, as expected when the target is a different functional form. Dark cells are states where no rule fires. **Right:** the coverage audit over the full four-dimensional input space. The hand design has no undefined state; four of five tuned controllers do.

TABLE IV

PER-RUN AUDIT OF THE TUNED CONTROLLERS. ACCURACY IMPROVED IN EVERY ONE OF THE FIVE RUNS; NOT ONE PRESERVED BOTH STRUCTURAL PROPERTIES THAT THE HAND-DESIGNED CONTROLLER HAS. “COVERAGE” IS THE PERCENTAGE OF 50,000 UNIFORMLY SAMPLED INPUT STATES THAT ACTIVATE NO RULE AT ALL AND THEREFORE LEAVE THE ACTUATOR WITHOUT A COMMAND; “ORDERING” IS WHETHER EVERY VARIABLE’S FUZZY SETS ARE STILL IN THEIR ORIGINAL LEFT-TO-RIGHT ORDER.

	test error	states with no output	ordering intact	verdict
Hand design	0.13299	0.00%	yes	both properties hold
Run 1	0.09682	0.39%	no	fails both
Run 2	0.08504	0.80%	yes	loses coverage
Run 3	0.08843	0.00%	no	loses ordering
Run 4	0.08980	0.37%	no	fails both
Run 5	0.08810	0.42%	yes	loses coverage
Runs preserving both properties				0 of 5

TABLE V
THE CHROMOSOME UNDER MAMDANI AND THREE SUGENO FORMULATIONS.

Formulation	Params	Bits	Auditable?
Mamdani (as built)	81	648	yes
Zero-order Sugeno	59	472	partly
First-order Sugeno	320	2,560	no
First-order + least squares	50	400	no

the safety property would need re-establishing. “Never heat a hot room” currently follows from the *structure* of the clamped FAM. With free linear consequents an optimiser could produce coefficients that violate it while still scoring well on average error, so the property would have to be reimposed as a constraint or a fitness penalty — and a penalty only discourages violations rather than preventing them, which is strictly weaker than what the Mamdani system provides. Section V-F is a demonstration of how much weaker.

Sugeno offers a computationally attractive alternative. For

this assistive-care scenario, it would need explicit behavioural constraints and a suitable audit before an accuracy gain could justify replacing the inspectable Mamdani design.

VI. OPTIMISER COMPARISON ON CEC'2005 FUNCTIONS

A. Functions and reproducible instances

Two functions from the CEC'2005 real-parameter suite [10] provide contrasting search landscapes. F6 is shifted Rosenbrock, a non-separable problem with a curved valley; F9 is shifted Rastrigin, a separable problem with many local minima. Their definitions are

$$F_6(\mathbf{x}) = \sum_{i=1}^{D-1} [100(z_i^2 - z_{i+1})^2 + (z_i - 1)^2] + 390, \quad (5)$$

$$F_9(\mathbf{x}) = \sum_{i=1}^D [z_i^2 - 10 \cos(2\pi z_i) + 10] - 330. \quad (6)$$

For F6, $\mathbf{z} = \mathbf{x} - \mathbf{o} + 1$, with domain $[-100, 100]^D$ and optimum 390. For F9, $\mathbf{z} = \mathbf{x} - \mathbf{o}$, with domain $[-5, 5]^D$ and optimum -330 . Both use the first D entries of the *official* CEC2005 shift files, matching the reference MATLAB implementation. The vendored data include source attribution and checksums; earlier experiments using locally generated shifts are archived and are not the results reported here. Figure 26 visualises the two official two-dimensional landscapes.

There are 15 independent repetitions for each algorithm, function and dimension, with $D \in \{2, 10\}$: 180 runs in total. Every run receives $10,000D$ objective evaluations. The random stream is determined by the repetition, algorithm, dimension and function; streams are reproducible but do not represent identical initial populations across different algorithms. An objective wrapper counts evaluations and rejects calls after the budget is exhausted. The two functions complement each other, but success on them does not establish general optimiser superiority across the CEC suite.

B. Algorithms and evaluation

The **GA** is real-coded, with population 50, tournament size three, blend crossover ($\alpha = 0.5$) at probability 0.9, Gaussian mutation with scale 0.1 times the search range and probability 0.1, and two-member elitism. Real coding avoids discretising these continuous benchmarks; it differs from the deliberately binary chromosome in the controller experiment.

Particle swarm optimisation (PSO) [11] uses a swarm of 40 particles, global-best topology, inertia decreasing linearly from 0.9 to 0.4, and cognitive/social coefficients 1.49445. Velocities are capped at 0.2 times the domain range and positions are bounded. Its observed poor runs should not be read as a limit on other PSO variants or attributed to absent velocity control: this implementation already has that control.

Simulated annealing (SA) [12] uses a single candidate with stochastic proposals and temperature-dependent acceptance of worse moves. Temperature decreases by a factor of 0.995 after each chain of 20 proposals; the Gaussian proposal scale decreases from 0.20 to 0.002 times the range. Initial temperature is calibrated to accept a typical uphill move with probability

0.5, and there are no restarts. These settings are stored with the results. Matching function evaluations equalises objective-call budgets, not wall-clock time or the suitability of a parameter setting to each function.

Table VI reports means, sample standard deviations, best and worst final objective values. Convergence plots show median best-so-far error with an interquartile band, and Fig. 20 displays all final-run errors. We also apply two-sided Mann–Whitney tests with Holm correction across the 12 pairwise comparisons. These test distributional differences; they are not tests of mean performance, and median differences alone do not fully describe skewed distributions.

C. Results and interpretation

On **F6 at $D = 2$** , PSO reaches the optimum to numerical precision and has the smallest mean error. GA's mean value is 396.728, slightly worse than SA's 396.145, because one poor GA run raises its mean. Yet GA's median is 390.025 versus SA's 394.243, and the distributions differ after correction ($p_{\text{Holm}} = 0.0045$). Mean and typical-run comparisons therefore answer different questions even in two dimensions. PSO differs from both alternatives after correction ($p_{\text{Holm}} < 0.001$).

On **F9 at $D = 2$** , both GA and PSO reach values extremely close to -330 . Their corrected comparison does not meet the 0.05 threshold ($p_{\text{Holm}} = 0.0559$), and their practical difference is negligible at the displayed precision. Both improve substantially on SA's typical final error, although SA also gets close to the optimum in some runs.

On **F6 at $D = 10$** , the ranking is particularly sensitive to summary choice. GA has the lowest mean objective, 465.167, while PSO's mean is approximately 4.03×10^5 . However, PSO's median is 391.661, lower than GA's 468.858. Poor PSO runs dominate its mean and spread. The GA–PSO distributional comparison is not significant after correcting all 12 tests ($p_{\text{Holm}} = 0.0559$). This does not establish equivalence: it shows why a claim that GA is unconditionally superior would exceed the evidence. SA's median is 750.341, and its distribution differs from both GA and PSO after correction.

On **F9 at $D = 10$** , GA is best on both mean and median error: mean error above the optimum is 0.00760, compared with 4.968 for PSO and 19.557 for SA. All three pairwise comparisons remain significant after correction. The advantage is clearer here than on Rosenbrock because it is not explained by a few extreme failures in one competitor.

These observations concern the implemented algorithms and fixed parameter settings. Tuning the annealing schedule, changing PSO's velocity cap or adding restarts, or changing GA variation could alter the outcomes and would constitute a new experiment. The benchmark does not reproduce the membership-function constraints or inference cost of the controller; its rankings cannot simply be transferred to the assistive-care tuning problem. Runtime also differs across algorithms, so equal evaluation budgets should be interpreted as search-effort control, not equal elapsed time.

TABLE VI

OFFICIAL CEC2005 INSTANCES: FINAL OBJECTIVE VALUES OVER 15 RUNS PER CELL. LOWER IS BETTER. STANDARD DEVIATION USES $n - 1$. OPTIMA ARE 390 (F6) AND -330 (F9).

Function	D	Algorithm	Mean	Std. dev.	Best	Worst
F6	2	GA	396.728071	20.776039	390.000003	469.881943
F6	2	PSO	390.000000	0.000000	390.000000	390.000000
F6	2	SA	396.144878	6.539262	390.305091	410.602213
F6	10	GA	465.166890	86.859403	395.757002	715.564526
F6	10	PSO	4.031×10^5	8.337×10^5	390.237147	2.014×10^6
F6	10	SA	1506.071423	2048.856085	428.638385	7670.107816
F9	2	GA	-330.000000	0.000000	-330.000000	-330.000000
F9	2	PSO	-330.000000	0.000000	-330.000000	-330.000000
F9	2	SA	-329.976837	0.047067	-329.999955	-329.844730
F9	10	GA	-329.992401	0.005209	-329.997721	-329.981570
F9	10	PSO	-325.031999	5.041317	-329.005041	-311.751448
F9	10	SA	-310.443145	10.497144	-322.738317	-288.878826

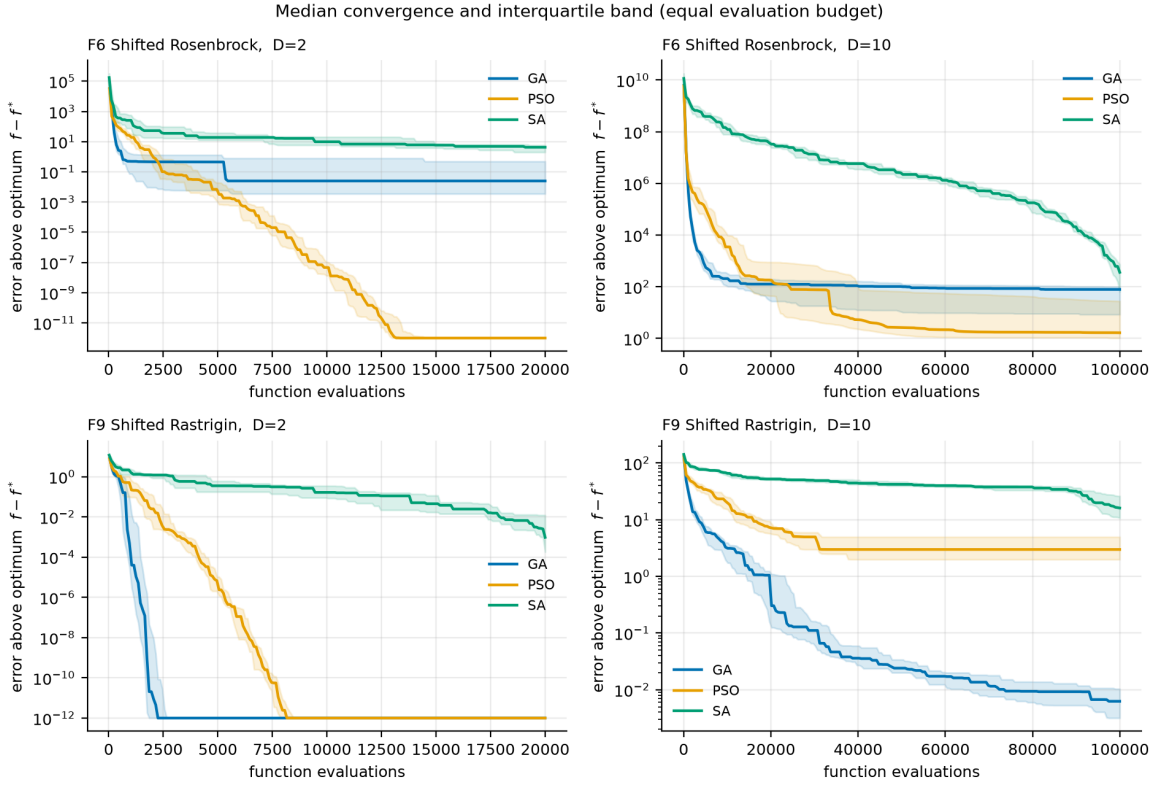


Fig. 19. Official CEC2005 instances: median best-so-far error and interquartile band over 15 runs. Each cell uses the same evaluation budget across algorithms. Errors below 10^{-12} are displayed at that floor for the logarithmic axis.

VII. CONCLUSIONS

The Mamdani controller provides a demonstrable mapping from room conditions and user preferences to heating/cooling and lighting actions. Membership functions, a linguistic rule base, rule-activation examples and response surfaces make the design inspectable. The reference controller and MATLAB implementation support reproducible examination of the chosen scenario; simulation alone does not establish effectiveness for real residents.

Genetic tuning reduces normalised test error by 32.6% relative to the hand-designed baseline on the synthetic reference-policy dataset. It also exposes a weakness of error-only optimisation: three of five tuned controllers reorder linguistic sets, four leave some inputs without a defined output, and none preserves both audited properties. The lower-error controller should therefore be treated as an experimental result rather than a deployment-ready improvement. A stronger next design would enforce coverage and ordering as feasibility constraints, then compare error among valid controllers using the same

Part 3: every one of the 180 runs, with the per-cell median marked

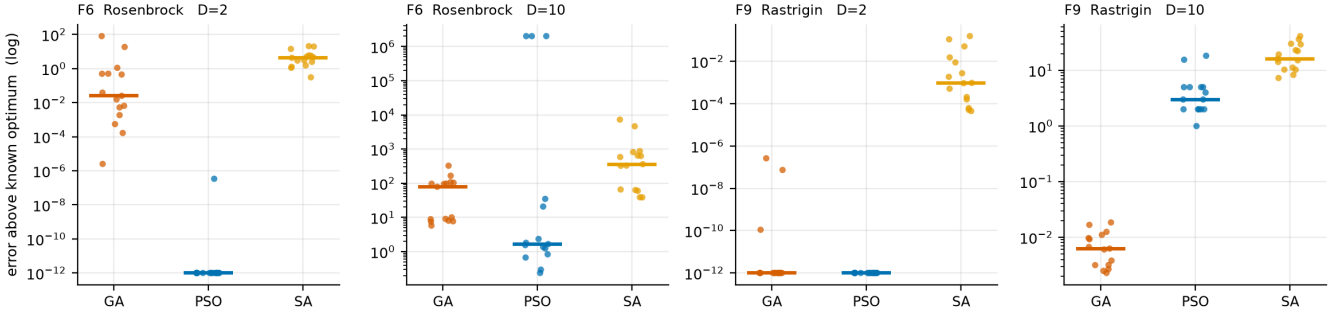


Fig. 20. All 180 final-run errors with per-cell medians. The large PSO tail on F6 at $D = 10$ explains the disagreement between mean and median rankings. A 10^{-12} plotting floor is used for errors at numerical precision.

training/test split.

The official CEC benchmark comparison reinforces the need to inspect distributions. PSO is strong in two dimensions; GA gives the lowest mean objective in both ten-dimensional cells, but the F6 median and tail tell a different story. Corrected rank tests qualify claims that would appear decisive from means alone. No universal winner follows from two functions, two dimensions and 15 repetitions.

The central limitations are the synthetic target policy, absence of real-resident evaluation, small number of controller-tuning runs and fixed optimiser parameters. The work demonstrates the requested design, optimisation and comparison processes while making the consequences of their assumptions visible.

INDIVIDUAL CONTRIBUTIONS

Bibek Paudyal (250288) coordinated the fuzzy-controller design and Python implementation. Sajan Mahat (250289) coordinated genetic tuning, the matched control and the coverage and linguistic-ordering audits. Siddhartha Bhatta (250620) coordinated the MATLAB implementation and benchmark comparison. All three members contributed equally to report preparation and review of the implementation, experiments and conclusions, with shared responsibility for the final submission.

REFERENCES

- [1] L. A. Zadeh, "Fuzzy sets," *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.
- [2] E. H. Mamdani and S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," *International Journal of Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.
- [3] T. Takagi and M. Sugeno, "Fuzzy identification of systems and its applications to modeling and control," *IEEE Trans. Systems, Man, and Cybernetics*, vol. SMC-15, no. 1, pp. 116–132, 1985.
- [4] C. C. Lee, "Fuzzy logic in control systems: Fuzzy logic controller — Parts I and II," *IEEE Trans. Systems, Man, and Cybernetics*, vol. 20, no. 2, pp. 404–435, 1990.
- [5] B. Kosko, *Neural Networks and Fuzzy Systems: A Dynamical Systems Approach to Machine Intelligence*. Englewood Cliffs, NJ: Prentice Hall, 1992.
- [6] W. Van Leekwijck and E. E. Kerre, "Defuzzification: Criteria and classification," *Fuzzy Sets and Systems*, vol. 108, no. 2, pp. 159–178, 1999.

- [7] O. Cordon, F. Gomide, F. Herrera, F. Hoffmann, and L. Magdalena, "Ten years of genetic fuzzy systems: Current framework and new trends," *Fuzzy Sets and Systems*, vol. 141, no. 1, pp. 5–31, 2004.
- [8] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA: Addison-Wesley, 1989.
- [9] J.-S. R. Jang, "ANFIS: Adaptive-network-based fuzzy inference system," *IEEE Trans. Systems, Man, and Cybernetics*, vol. 23, no. 3, pp. 665–685, 1993.
- [10] P. N. Suganthan, N. Hansen, J. J. Liang, K. Deb, Y.-P. Chen, A. Auger, and S. Tiwari, "Problem definitions and evaluation criteria for the CEC 2005 special session on real-parameter optimization," Nanyang Technological Univ., Singapore, Tech. Rep., 2005.
- [11] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Networks*, 1995, pp. 1942–1948.
- [12] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

APPENDIX A

SOURCE CODE AND REPRODUCTION

The complete implementation — the controller, the genetic tuning experiments, the benchmark study, the MATLAB port and every script that produced a figure or a number in this paper — is available at:

<https://github.com/bpdyl/ai-research-topic-modeling>

Task 2 lives under `assistive-care-flc/` and reproduces from a clean checkout. Three drivers regenerate the experiments in order: `run_part1.py` (the controller and its structural audit), `run_part2.py` (genetic tuning with its budget-matched control) and `run_part3.py` (the 180 benchmark runs). A fourth, `make_report_figures.py`, derives the remaining figures and the two audits of Section V-F from the stored results without re-running any experiment.

The MATLAB system is generated from the controller definition used by the reference engine; parity is checked on selected examples rather than assumed. It requires no Fuzzy Logic Toolbox and runs on base MATLAB. All random seeds are fixed, and an automated test suite pins the behavioural claims made in this paper, including the safety property that the controller never heats a hot room.

APPENDIX B

SUPPORTING FIGURES

The figures below support claims made in the body of the paper. Each one substantiates a statement the body makes in

a sentence, and is placed here so that the argument reads continuously without a full-page figure interrupting it. Every figure is regenerated by the scripts listed in Appendix A from stored results, with the benchmark panels regenerated for the official-shift experiment.

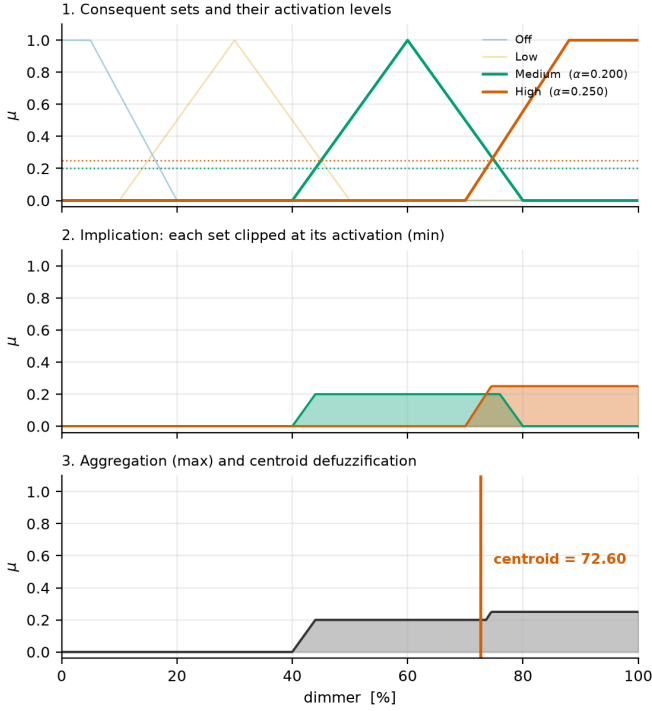


Fig. 21. Implication, aggregation and defuzzification for the dimmer output on the worked scenario — the counterpart to Fig. 10, which shows the same three stages for hvac. Two lighting rules fire: Dark $\wedge$ Active $\rightarrow$ High at $\alpha = 0.25$, and Dim $\wedge$ Active $\rightarrow$ Medium at $\alpha = 0.20$. Each consequent set is clipped at its own α (the flat tops), the two are unioned, and the centroid of the result is 72.60%. Note that the higher-activation rule is the one demanding *more* light, which is why the crisp result sits well above the midpoint of the two sets rather than between them.

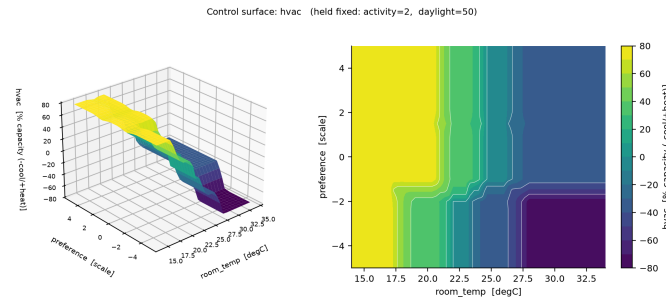


Fig. 22. Control surface: hvac against room_temp and preference, at light activity. This is the slice that shows the clamped modifier of (1) doing its work. Preference shifts the surface bodily up and down — a resident who prefers warmth is given a warmer room at every temperature — but it never inverts the slope in temperature. A resident who sets the dial to +5 therefore cannot cause a hot room to be heated, which is the safety property the inner clip guarantees by construction.

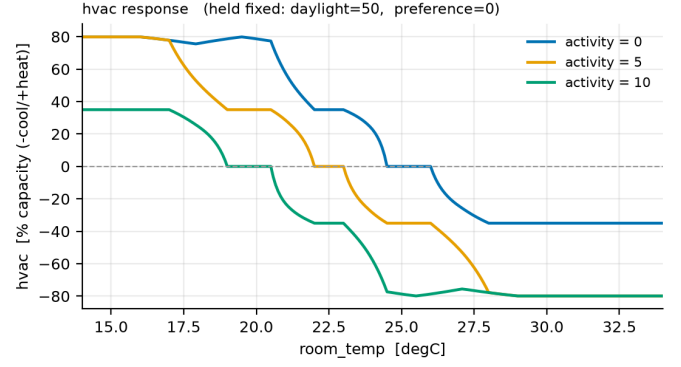


Fig. 23. HVAC command against room temperature at three activity levels, at neutral preference. Three things are visible. The curves are ordered as the design intends: a resting occupant is given a warmer room than an active one at every temperature, which is the metabolic-heat argument of Section I realised in the rule base. The response saturates symmetrically at roughly $\pm 79.9\%$, the centroid limit quantified in Section IV. And the small non-monotonicity noted there — about 0.6% of the output span — is visible as a shallow reversal near the points where the demand scale saturates and two adjacent temperature sets come to share a consequent.

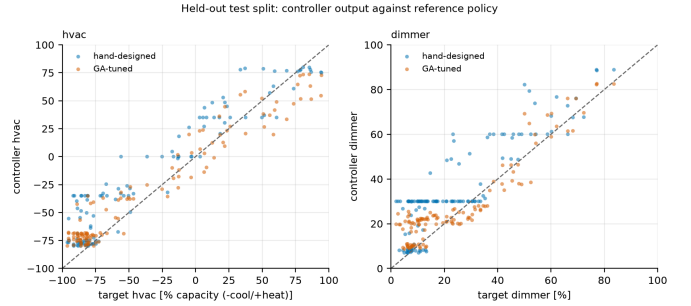


Fig. 24. Controller output against the reference policy on the 120-example held-out split, before and after tuning, for both actuators. The tuned points lie visibly closer to the diagonal. The residual spread is expected rather than a failure: the reference policy is a saturating linear ramp and the controller is a union of clipped fuzzy sets, so the two functional forms cannot coincide exactly. That mismatch is deliberate — had the target been generated by a Mamdani system with the same rule base, the error could in principle have been driven to zero, which would have demonstrated only that the encoding can represent its own output.

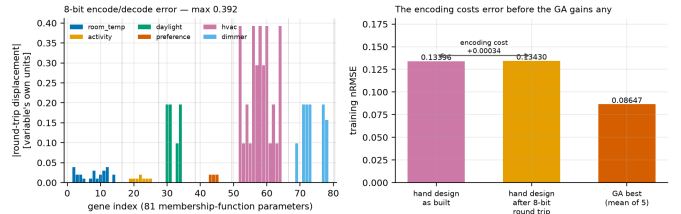


Fig. 25. What the 8-bit-per-gene binary encoding costs before the search begins. **Left:** the round-trip displacement of each of the 81 membership-function parameters after encoding the hand-designed controller and decoding it again — at most 0.392 units, which is half a quantisation step on the widest universe. **Right:** the training error that displacement causes, +0.000342 in normalised RMSE, set against the improvement the search subsequently wins. The encoding costs roughly 0.7% of what the search gains, which is the evidence for the claim in Section V that quantisation is not what limits the result.

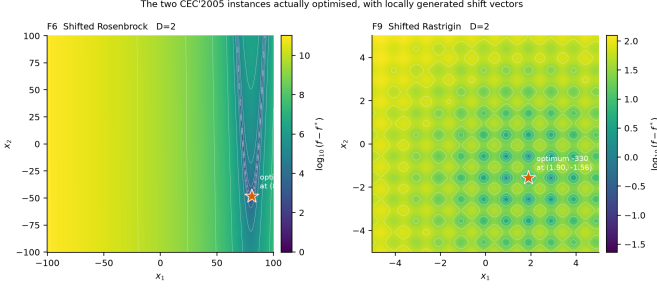


Fig. 26. The two CEC’2005 instances actually optimised, at $D = 2$, plotted as $\log_{10}(f - f^*)$ so that structure is visible on both scales at once. F6’s long, narrow, curved valley and F9’s regular lattice of local minima are the complementary difficulties described in Section VI: one is hard because the descent direction keeps turning, the other because there are 10^D basins to be trapped in. The marked optima confirm the shift vectors are doing their job — neither optimum sits at the centre of its domain, so an algorithm that initialises symmetrically about the origin gains no free advantage. The shift vectors are taken from the official CEC2005 distribution.

APPENDIX C MATLAB BENCHMARK IMPLEMENTATION

The following function implements both selected benchmark equations using the vendored official shift files. Its Python counterpart was checked against the same published

data and known optima. This additional MATLAB listing was not executed in the report-revision environment.

```
function values = cec2005_f6_f9(X, function_number)
% CEC2005 F6/F9 with official shift data, first D
% coordinates.
% Equations follow Suganthan et al. (2005); data provenance
% is in
% ../data/cec2005/provenance.json. Rows of X are candidate
% solutions.
data_dir = fullfile(fileparts(mfilename('fullpath'))), '..
    ', 'data', 'cec2005');
D = size(X, 2);
assert(D >= 2 && D <= 100, 'Supported_dimensions:_2_
    through_100');
if function_number == 6
    shift = readmatrix(fullfile(data_dir, '
        rosenbrock_func_data.txt'));
    Z = X - reshape(shift(1:D), 1, D) + 1;
    values = sum(100 .* (Z(:,1:end-1).^2 - Z(:,2:end)).^2
        ...
        + (Z(:,1:end-1) - 1).^2, 2) + 390;
elseif function_number == 9
    shift = readmatrix(fullfile(data_dir, '
        rastrigin_func_data.txt'));
    Z = X - reshape(shift(1:D), 1, D);
    values = sum(Z.^2 - 10 .* cos(2*pi*Z) + 10, 2) - 330;
else
    error('Only_CEC2005_functions_6_and_9_are_implemented_
        here');
end
end
```